1. MotrixLab 强化学习进阶知识点总结（第二周）

> **使用说明**：本篇重点梳理 MotrixLab 的工程化核心机制，涵盖配置系统、注册系统与训练工作流。建议在实际进行环境搭建与代码实操时，对照本指南进行核对。

⚙️ 一、配置系统：环境配置与训练配置协作机制

1. 配置系统的核心思想

避免把大量参数（学习率、环境数量、地形、奖励权重等）硬编码在代码中。

- **核心理念**：`Code defines logic, Configuration defines experiment.`（代码负责实现算法逻辑，配置负责决定当前实验）。

2. 双轨架构：EnvCfg 与 RLCfg

一个机器人实验主要由两部分配置组成，二者的核心协作闭环是：**EnvCfg 提供数据，RLCfg 利用数据完成策略更新。**

> 💡 **一句话总结：EnvCfg 决定世界，RLCfg 决定学习。**

- **EnvCfg（决定环境与任务）**：
  - `Observation`：决定Policy Network的输入（如 `base_lin_vel`、`joint_pos`）。
  - `Action`：决定机器人作用于环境的方式（如关节位置、速度、力矩）。
  - `Reward`：定义机器人应该学会什么（如跟踪奖励、能量消耗惩罚）。
  - `Termination`：决定 Episode 何时结束（如摔倒、超时）。
  - `Randomization`：域随机化（质量、摩擦力等），用于减少 Sim-to-Real Gap。
  - `Curriculum`：课程学习，逐步增加难度（如 平地 → 缓坡 → 崎岖 → 障碍）。
- **RLCfg（决定算法与训练）**：
  - `PPO Hyperparameters`：控制优化过程与收敛（如 `learning_rate`、`gamma`、`lam`、`clip_param`、`entropy_coef`）。
  - `Runner`：控制训练全局流程（如 `num_steps_per_env`、`max_iterations`）。
  - `Logging & Checkpoint`：控制日志记录工具（TensorBoard/W&B）与模型保存频率。

3. 常见参数归类速查表

所属配置类	典型参数示例	作用域
EnvCfg (环境)	`model_file`, `sim_dt`, `render_spacing`, `max_episode_seconds`	物理仿真、时间步长、渲染、任务定义
RLCfg (算法)	`learning_rate`, `num_envs`, `rollouts`, `ratio_clip`	神经网络优化、并行环境采样、PPO 超参数

🔗 二、注册系统：新环境接入框架指南

1. Registry（注册系统）的作用

建立 Task Name 与配置之间的映射关系，统一管理所有任务，支持框架根据 Task Name 自动加载对应的环境和配置，而无需修改框架主体代码。

2. 注册系统工作流与核心注册项

> **工作流**：读取 Task Name → Registry 匹配 → 加载 Environment & EnvCfg & RLCfg → 创建训练环境。
> *(注：缺少任何一部分，都无法启动训练。)*

三个核心注册项：

- **注册 Environment**：告诉框架环境的实现位置（负责 `reset`、`step`、`reward`、`obs`）。
- **注册 EnvCfg**：描述 *Robot lives in what world.* (如 `env_cfg_entry_point=HumanoidEnvCfg`)
- **注册 RLCfg**：描述 *Robot learns how.* (如 `rl_cfg_entry_point=HumanoidPPORunnerCfg`)

3. 新建 Task 的标准步骤与代码规范

每个任务目录通常包含：`__init__.py`、`env_cfg.py`、`rl_cfg.py`、`xxx_env.py`。

开发新任务的 7 步法：

1. 复制已有 Task 模板（如 `humanoid`）。
2. 修改 Task Name。
3. 创建新的 Environment（必须实现三个抽象方法：`apply_action()`、`update_state()`、`reset()`）。
4. 创建新的 EnvCfg。
5. 创建新的 RLCfg。
6. 修改相关代码，使用装饰器完成注册：
   - `@registry.envcfg("task_name")`
   - `@registry.env("task_name", "np")`
   - `@rlcfg("task_name")`
7. 梳理 Import 链：确保任务目录的 `__init__.py` 导入了环境；分类目录的 `__init__.py` 导入了任务。

🚀 三、训练流程：从启动脚本到模型保存

1. 训练启动与参数解析

命令行示例：`python train.py --task humanoid`

> **核心逻辑**：Train Script controls workflow, Registry loads components.（训练脚本负责组织流程，注册系统负责自动加载任务）。

常见覆盖参数（可直接在命令行覆盖默认配置）：

- `--task`：指定任务名。
- `--num_envs`：并行环境数量。

- `--seed`：控制随机种子。
- `--max_iterations`：最大迭代次数。
- `--headless`：无头模式（不渲染画面，加速训练）。

2. 训练全局架构流转

> Command Line → `train.py` → Parse Arguments → Registry → Load Task → EnvCfg / RLCfg → Create Environment (Robot, Terrain, Sensors 等) → Create Runner (Policy, Optimizer, Buffer 等) → Start Training。

3. 日志、输出与实操验证

训练过程中，框架会按照设定的 `save_interval` 自动保存状态，并持续输出数据。

输出目录结构 (`runs/xxx/`)：

- `checkpoints/`：包含 `model_X.pt`（保存内容：Policy、Optimizer、Runner State），用于恢复训练或推理部署。
- `logs/`：包含 `events.out.tfevents...`（TensorBoard 日志），记录 Loss、Reward、Episode Length 等。
- `videos/`：Rollout 视频、可视化结果。
- `configs/`：记录当前的 EnvCfg 和 RLCfg，便于实验复现。

实操闭环验证要点：

1. **启动 TensorBoard**：`uv run tensorboard --logdir runs/xxx`。重点观察 `Loss/value`、`Loss/surrogate`、`Loss/entropy` 及 `Train/mean_reward` 曲线判断收敛情况。
2. **推理验证 (Play)**：执行 `play.py` 脚本（指定环境、策略后端和具体模型 checkpoint 路径），验证环境与策略网络能否正确闭环推理。

🌟 四、第二周核心主线总结

1. **配置分离**：EnvCfg 决定世界，RLCfg 决定学习；代码定义逻辑，配置决定实验。
2. **注册映射**：Registry 建立 Task Name 到 Environment、EnvCfg、RLCfg 的映射，实现自动化、解耦的加载。
3. **训练闭环**：命令行 → 解析参数 → 注册系统加载任务 → 创建环境与 Runner → 训练 → 保存 Checkpoint 与日志。
4. **实操闭环**：修改配置 → 启动训练 → 观测 TensorBoard 曲线分析瓶颈 → 加载 Checkpoint 进行可视化推理验证。